

Библиотеки numpy и matplotlib

М. А. Ложников

29 апреля 2019 г.

Это предварительная невыверенная версия. Читайте на свой страх и риск.

1 Работа с массивами с помощью библиотеки numpy

1.1 Импорт библиотеки

Обычно модуль numpy импортируют так:

```
1 import numpy as np
```

В этом случае к функциям из этого модуля следует добавлять префикс `np` с точкой, например, `np.array()` и т. д.

1.2 Создание и индексация массива

```
1 a = np.array([1, 4, 5, 8], float) # Создаём массив из списка.
2 # Тип данных (второй параметр) можно не указывать.
3
4 # Массивы индексируются точно так же, как и списки
5 b = a[:2] # b = [1., 4.]
6 c = a[3]  # c = 8.;
7 a[0] = 5  # Меняем самый первый элемент массива a на 5
```

Кроме того, массив можно загрузить из текстового файла.

Если массив одномерный, то его элементы должны располагаться в одной строке текстового файла через пробел либо по одному элементу в строке.

Двумерный массив (матрица) размера $N \times M$ должен быть задан следующим образом: в i -ой строке текстового файла, $i = 1, \dots, N$, должны располагаться ровно M чисел через пробел.

```
1 a = np.loadtxt('array.txt')
```

и сохранить в текстовый файл

```
1 np.savetxt('array.txt', a)
```

Существуют также функции, позволяющие загружать и сохранять массив в бинарном формате, они работают намного быстрее.

Массивы могут быть многомерными, вот пример матрицы (двумерный массив).

```
1 # Матрицу можно создать из двумерного списка
2 >>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
3 >>> a
4 array([[ 1.,  2.,  3.],
5        [ 4.,  5.,  6.]])
6 >>> a[0, 0] # Матрица индексируется номером строки и номером столбца через запятую
7 1.0        # в одних квадратных скобках. Строки и столбцы ну
8 >>> a[0, 1]
9 2.0
```

В примерах этого раздела будем считать, что весь код вводится в интерпретатор Python в интерактивном режиме, символы >>> означают приглашение для ввода, выводимое интерпретатором, после чего следует введенная пользователем команда. Команда в результате своей работы может вывести что-либо на экран (такие строки не содержат символов >>> в начале). Для читабельности некоторые команды отделены друг от друга пустыми строками.

Срезки используются точно так же как и со списками и берутся отдельно по разным измерениям.

```
1 >>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
2 >>> a[1, :]
3 array([ 4.,  5.,  6.]) # Последняя строка матрицы
4 >>> a[:, 2] # Использование : указывает на то, что нужно взять всё измерение целиком
5 array([ 3.,  6.])      # Последний столбец матрицы (одномерный массив)
6
7 >>> a[-1, -2:]          # Если индексы отрицательные, то это значит, что эти числа следует
8                        # вычесть из размера массива по соответствующему измерению
9 array([ 5.,  6.])      # На выходе одномерный массив т.к. указана только одна строка.
10
11 >>> a[-1:, -2:]         # В этом примере на выходе будет матрица даже в том случае,
12 array([[ 5.,  6.]])    # если она состоит из одной строки (столбца) поскольку по обоим
13                        # направлениям указаны срезы.
```

Размеры матрицы хранятся в кортеже `shape`

```
1 >>> a.shape
2 (2, 3)
```

Функция `len` возвращает размер массива по первой координате

```
1 >>> a = np.array([[1, 2, 3], [4, 5, 6]], float) # Матрица 2*3
2 >>> len(a)
3 2
```

Для проверки принадлежности массиву используется ключевое слово `in`. С массивами чисел с плавающей точкой лучше не использовать.

```
1 >>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
2 >>> 2 in a
3 True
4 >>> 0 in a
5 False
```

Элементы массива можно перегруппировать, например,

```
1 >>> a = np.array(range(10), float)
2 >>> a
3 array([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.])
4 >>> b = a.reshape((5, 2)) # Функция принимает кортеж с размерностями нового массива
5 >>> b                       # Важно: массивы a и b ссылаются на одни и те же данные.
6 array([[ 0.,  1.],
7        [ 2.,  3.],
8        [ 4.,  5.],
9        [ 6.,  7.],
10       [ 8.,  9.]])
11 >>> b.shape
12 (5, 2)
```

Функция `reshape()` возвращает массив указанной формы, полученный из другого массива, при этом она не модифицирует старый. Однако, оба массива будут ссылаться на одни и те же данные, то есть при изменении элементов одного массива, поменяются и соответствующие элементы второго. Для полного (глубокого) копирования массива следует использовать функцию `copy()`. Иначе python может просто создать ссылку.

```
1 >>> a = np.array([1, 2, 3], float)
2 >>> b = a
3 >>> c = a.copy()
4 >>> a[0] = 0
5 >>> a
6 array([0., 2., 3.])
7 >>> b
8 array([0., 2., 3.])
9 >>> c
10 array([1., 2., 3.])
```

1.2.1 Создание списка из массива

```
1 >>> a = np.array([1, 2, 3], float)
2 >>> a.tolist()
3 [1.0, 2.0, 3.0]
4 >>> list(a)
5 [1.0, 2.0, 3.0]
```

1.2.2 Преобразование numpy массива в сырой массив байт

```
1 >>> a = array([1, 2, 3], float)
2 >>> s = a.tostring() # Преобразование в бинарный формат
3 >>> s
4 b'\x00\x00\x00\x00\x00\x00\xf0?\x00\x00\x00\x00\x00\x00@'
5 >>> np.fromstring(s) # Обратное преобразование
6 array([ 1.,  2.,  3.]
```

1.2.3 Заполнение массива одним значением

```
1 >>> a = array([1, 2, 3], float)
2 >>> a
3 array([ 1.,  2.,  3.])
4 >>> a.fill(0)
5 >>> a
6 array([ 0.,  0.,  0.]
```

1.2.4 Транспонирование матрицы

```
1 >>> a =
2 np.array(range(6), float).reshape((2, 3))
3 >>> a
4 array([[ 0.,  1.,  2.],
5        [ 3.,  4.,  5.]])
6 >>> a.transpose()
7 array([[ 0.,  3.],
8        [ 1.,  4.],
9        [ 2.,  5.]])
```

1.2.5 Схлопывание многомерного массива в одномерный

```
1 >>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
2 >>> a
3 array([[ 1.,  2.,  3.],
4        [ 4.,  5.,  6.]])
5 >>> a.flatten()
6 array([ 1.,  2.,  3.,  4.,  5.,  6.]
```

1.2.6 Объединение массивов

```
1 >>> a = np.array([1,2], float)
2 >>> b = np.array([3,4,5,6], float)
3 >>> c = np.array([7,8,9], float)
4 >>> np.concatenate((a, b, c))
5 array([1., 2., 3., 4., 5., 6., 7., 8., 9.]
```

1.2.7 Объединение массивов вдоль определённого направления

```
1 >>> a = np.array([[1, 2], [3, 4]], float)
2 >>> b = np.array([[5, 6], [7,8]], float)
3 >>> np.concatenate((a,b))
4 array([[ 1.,  2.],
5        [ 3.,  4.],
6        [ 5.,  6.],
7        [ 7.,  8.]])
8 >>> np.concatenate((a,b), axis=0)
9 array([[ 1.,  2.],
10       [ 3.,  4.],
11       [ 5.,  6.],
12       [ 7.,  8.]])
13 >>> np.concatenate((a,b), axis=1)
14 array([[ 1.,  2.,  5.,  6.],
15       [ 3.,  4.,  7.,  8.]])
```

1.2.8 Увеличение размерности с помощью np.newaxis

```
1 >>> a = np.array([1, 2, 3], float)
2 >>> a
3 array([1., 2., 3.])
4 >>> a[:,np.newaxis]
5 array([[ 1.],
6        [ 2.],
7        [ 3.]])
8 >>> a[:,np.newaxis].shape
9 (3,1)
10 >>> b[np.newaxis,:]
11 array([[ 1.,  2.,  3.]])
12 >>> b[np.newaxis,:].shape
13 (1,3)
```

1.2.9 Другие способы создания массива

Функция `arange` —аналог функции `range`, только возвращает массив, а не список

```
1 >>> np.a
2 range(5, dtype=float)
3 array([ 0.,  1.,  2.,  3.,  4.])
4 >>> np.arange(1, 6, 2, dtype=int)
5 array([1, 3, 5])
```

Матрица из нулей, единиц, тождественная матрица и матрица из единиц на k -ой диагонали

```
1 >>> np.ones((2,3), dtype=float)
2 array([[ 1.,  1.,  1.],
3        [ 1.,  1.,  1.]])
4 >>> np.zeros(7, dtype=int)
5 array([0, 0, 0, 0, 0, 0, 0])
6 >>> np.identity(4, dtype=float)
7 array([[ 1.,  0.,  0.,  0.],
8        [ 0.,  1.,  0.,  0.],
9        [ 0.,  0.,  1.,  0.],
10       [ 0.,  0.,  0.,  1.]])
11 >>> np.eye(4, k=1, dtype=float)
12 array([[ 0.,  1.,  0.,  0.],
13        [ 0.,  0.,  1.,  0.],
14        [ 0.,  0.,  0.,  1.],
15        [ 0.,  0.,  0.,  0.]])
```

1.3 Операции с массивами

```
1 >>> a = np.array([1,2,3], float)
2 >>> b = np.array([5,2,6], float)
3 >>> a + b
4 array([6., 4., 9.])
5 >>> a - b
6 array([-4., 0., -3.])
7 >>> a * b
8 array([5., 4., 18.])
9 >>> b / a
10 array([5., 1., 2.])
11 >>> a % b
12 array([1., 0., 3.])
13 >>> b ** a
14 array([5., 4., 216.])
15 # Обратите внимание, что умножение - поэлементное
16 >>> a = np.array([[1,2], [3,4]], float)
17 >>> b = np.array([[2,0], [1,3]], float)
18 >>> a * b
19 array([[2., 0.], [3., 12.]])
```

К массивам можно применять поэлементно функции `abs`, `sign`, `sqrt`, `log`, `log10`, `exp`, `sin`, `cos`, `tan`, `arcsin`, `arccos`, `arctan`, `sinh`, `cosh`, `tanh`, `arcsinh`, `arccosh`, и `arctanh`, например,

```
1 >>> a = np.array([1, 4, 9], float)
2 >>> np.sqrt(a)
3 array([ 1.,  2.,  3.] )
```

Массивы можно округлять поэлементно

```
1 >>> a = np.array([1.1, 1.5, 1.9], float)
2 >>> np.floor(a)
3 array([ 1.,  1.,  1.])
4 >>> np.ceil(a)
5 array([ 2.,  2.,  2.])
6 >>> np rint(a)
7 array([ 1.,  2.,  2.])
```

Кроме того

```
1 >>> np.pi
2 3.1415926535897931
3 >>> np.e
4 2.7182818284590451
```

1.4 Базовые операции с массивами

1.4.1 Сумма и произведение всех эл-тов

```
1 >>> a = np.array([2, 4, 3], float)
2 >>> a.sum()
3 9.0
4 >>> a.prod()
5 24.0
6 >>> np.sum(a)
7 9.0
8 >>> np.prod(a)
9 24.0
```

1.4.2 Среднее и вариация

```
1 >>> a = np.array([2, 1, 9], float)
2 >>> a.mean()
3 4.0
4 >>>
5 a.var()
6 12.666666666666666
7 >>> a.std()
8 3.5590260840104371
```

1.4.3 Минимум и Максимум

```
1 >>> a = np.array([2, 1, 9], float)
2 >>> a.min()
3 1.0
4 >>> a.max()
```

```
5 9.0
6 >>> a = np.array([2, 1, 9], float)
7 >>> a.argmin()
8 1
9 >>> a.argmax()
10 2
```

1.4.4 Применение базовых операций вдоль выбранной оси

```
1 >>> a = np.array([[0, 2], [3, -1], [3, 5]], float)
2 >>> a.mean(axis=0)
3 array([ 2.,  2.])
4 >>> a.mean(axis=1)
5 array([ 1.,  1.,  4.])
6 >>> a.min(axis=1)
7 array([ 0., -1.,  3.])
8 >>> a.max(axis=0)
9 array([ 3.,  5.])
```

1.4.5 Сортировка массива

```
1 >>> a = np.array([6, 2, 5, -1, 0], float)
2 >>> sorted(a)
3 [-1.0, 0.0, 2.0, 5.0, 6.0]
4 >>> a.sort()
5 >>> a
6 array([-1.,  0.,  2.,  5.,  6.])
```

1.4.6 Ещё немного операций

```
1 >>> a = np.array([6, 2, 5, -1, 0], float)
2 >>> a.clip(0, 5)
3 array([ 5.,  2.,  5.,  0.,  0.])
4 >>> a = np.array([1, 1, 4, 5, 5, 5, 7], float)
5 >>> np.unique(a)
6 array([ 1.,  4.,  5.,  7.])
7 >>> a = np.array([[1, 2], [3, 4]], float)
8 >>> a.diagonal()
9 array([ 1.,  4.])
```

1.5 Операции сравнения

```
1 >>> a = np.array([1, 3, 0], float)
2 >>> b = np.array([0, 3, 2], float)
3 >>> a > b
4 array([ True, False, False], dtype=bool)
```



```
5 >>> a == b
6 array([False,  True, False], dtype=bool)
7 >>> a <= b
8 array([False,  True,  True], dtype=bool)
9 >>> a = np.array([1, 3, 0], float)
10 >>> a > 2
11 array([False,  True, False], dtype=bool)
```

1.5.1 Составные операции сравнения

```
1 >>> c = np.array([ True, False, False], bool)
2 >>> any(c)
3 True
4 >>> all(c)
5 False
```

Комбинирование производится с помощью специальных функций `logical_and`, `logical_or` и `logical_not`.

```
1 >>> a = np.array([1, 3, 0], float)
2 >>> np.logical_and(a > 0, a < 3)
3 array([ True, False, False], dtype=bool)
4 >>> b = np.array([True, False, True], bool)
5 >>> np.logical_not(b)
6 array([False,  True, False], dtype=bool)
7 >>> c = np.array([False, True, False], bool)
8 >>> np.logical_or(b, c)
9 array([ True,  True, False], dtype=bool)
```

1.5.2 Функция `where`

```
1 >>> a = np.array([1, 3, 0], float)
2 >>> np.where(a != 0, 1 / a, a)
3 # Первый параметр - условие
4 # Второй параметр - значение, если условие истинно
5 # Третий параметр - значение, если условие ложно
6 array([ 1. ,  0.33333333,  0. ])
7 >>> np.where(a > 0, 3, 2)
8 array([3, 3, 2])
```

1.6 Выборка элементов

1.6.1 Выборка элементов с помощью условий в квадратных скобках

Булевы маски

```
1 >>> a = np.array([[6, 4], [5, 9]], float)
2 >>> a >= 6
```

```
3 array([[ True, False],
4        [False,  True]], dtype=bool)
5 >>> a[a >= 6]
6 array([ 6.,  9.])
7 >>> a = np.array([[6, 4], [5, 9]], float)
8 >>> sel = (a >= 6)
9 >>> a[sel]
10 array([ 6.,  9.])
11 >>> a[np.logical_and(a > 5, a < 9)]
12 array([ 6.])
```

Массивы индексов

```
1 >>> a = np.array([2, 4, 6, 8], float)
2 >>> b = np.array([0, 0, 1, 3, 2, 1], int)
3 >>> a[b]
4 array([ 2.,  2.,  4.,  8.,  6.,  4.])
5 >>> a = np.array([2, 4, 6, 8], float)
6 >>> a[[0, 0, 1, 3, 2, 1]]
7 array([ 2.,  2.,  4.,  8.,  6.,  4.])
8 >>> a = np.array([[1, 4], [9, 16]], float)
9 >>> b = np.array([0, 0, 1, 1, 0], int)
10 >>> c = np.array([0, 1, 1, 1, 1], int)
11 >>> a[b,c]
12 array([ 1.,  4., 16., 16.,  4.])
```

1.6.2 Функция take делает всё то же самое

```
1 >>> a = np.array([2, 4, 6, 8], float)
2 >>> b = np.array([0, 0, 1, 3, 2, 1], int)
3 >>> a.take(b)
4 array([ 2.,  2.,  4.,  8.,  6.,  4.])
5 >>> a = np.array([[0, 1], [2, 3]], float)
6 >>> b = np.array([0, 0, 1], int)
7 >>> a.take(b, axis=0)
8 array([[ 0.,  1.],
9        [ 0.,  1.],
10       [ 2.,  3.]])
11 >>> a.take(b, axis=1)
12 array([[ 0.,  0.,  1.],
13       [ 2.,  2.,  3.]])
```

1.6.3 Функция put

```
1 >>> a = np.array([0, 1, 2, 3, 4, 5], float)
2 >>> b = np.array([9, 8, 7], float)
3 # Заменяем выбранные элементы массива a массивом b
4 >>> a.put([0, 3], b)
```

```
5 >>> a
6 array([ 9.,  1.,  2.,  8.,  4.,  5.])
7 >>> a = np.array([0, 1, 2, 3, 4, 5], float)
8 # Заменяем выбранные элементы массива a числом 5
9 >>> a.put([0, 3], 5)
10 >>> a
11 array([ 5.,  1.,  2.,  5.,  4.,  5.])
```

1.7 Матричная и векторная арифметика

1.7.1 Скалярное и матричное произведения

```
1 >>> a = np.array([1, 2, 3], float)
2 >>> b = np.array([0, 1, 1], float)
3 >>> np.dot(a, b)
4 5.0
5 >>> a = np.array([[0, 1], [2, 3]], float)
6 >>> b = np.array([2, 3], float)
7 >>> c = np.array([[1, 1], [4, 0]], float)
8 >>> a
9 array([[ 0.,  1.],
10        [ 2.,  3.]])
11 >>> np.dot(b, a)
12 array([ 6., 11.])
13 >>> np.dot(a, b)
14 array([ 3., 13.])
15 >>> np.dot(a, c)
16 array([[ 4.,  0.],
17        [14.,  2.]])
18 >>> np.dot(c, a)
19 array([[ 2.,  4.],
20        [ 0.,  4.]])
```

1.7.2 Внутреннее, внешнее и векторное произведения

```
1 >>> a = np.array([1, 4, 0], float)
2 >>> b = np.array([2, 2, 1], float)
3 >>> np.outer(a, b)
4 array([[ 2.,  2.,  1.],
5        [ 8.,  8.,  4.],
6        [ 0.,  0.,  0.]])
7 >>> np.inner(a, b)
8 10.0
9 >>> np.cross(a, b)
10 array([ 4., -1., -6.] )
```

1.7.3 Различные матричные операции

Определитель матрицы

```
1 >>> a = np.array([[4, 2, 0], [9, 3, 7], [1, 2, 1]], float)
2 >>> a
3 array([[ 4.,  2.,  0.],
4         [ 9.,  3.,  7.],
5         [ 1.,  2.,  1.]])
6 >>> np.linalg.det(a)
7 -53.999999999999993
```

Собственные значения матрицы

```
1 >>> vals, vecs = np.linalg.eig(a)
2 >>> vals
3 array([ 9. ,  2.44948974, -2.44948974])
4 >>> vecs
5 array([[ -0.3538921 , -0.56786837,  0.27843404],
6         [-0.88473024,  0.44024287, -0.89787873],
7         [-0.30333608,  0.69549388,  0.34101066]])
```

Обратная матрица

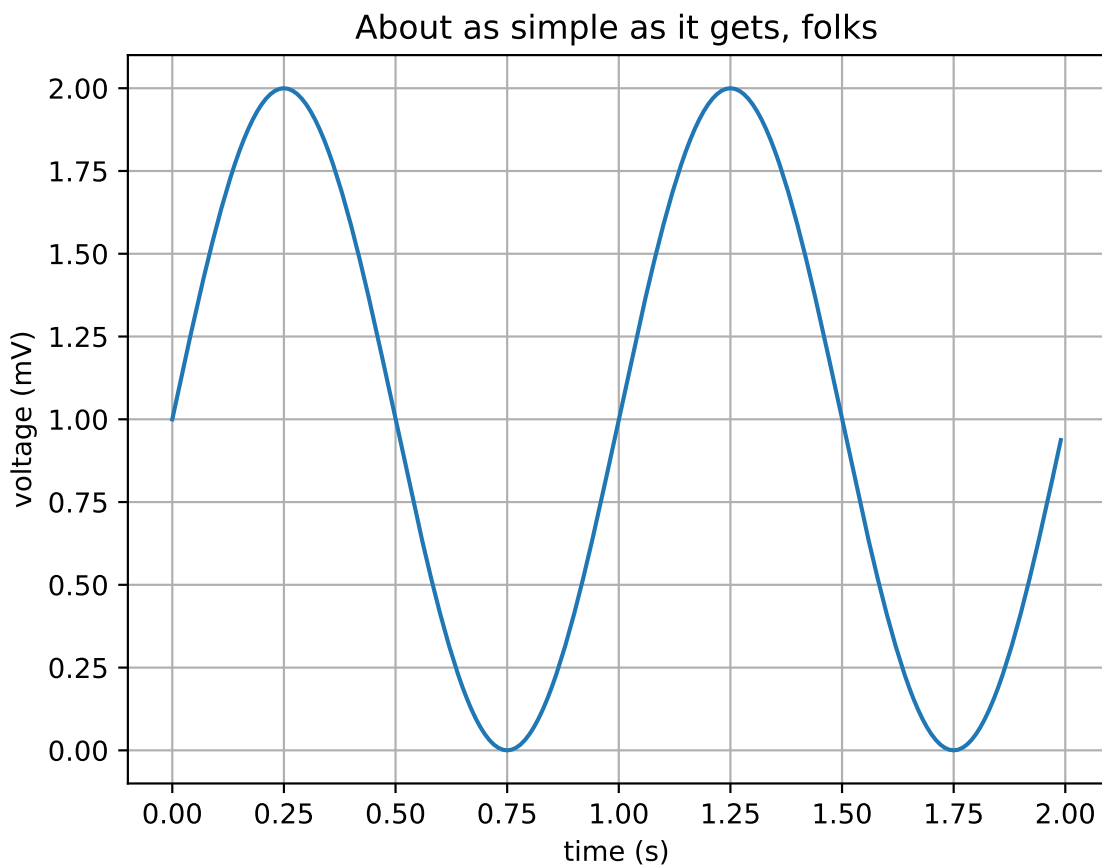
```
1 >>> b = np.linalg.inv(a)
2 >>> b
3 array([[ 0.14814815,  0.07407407, -0.25925926],
4         [ 0.2037037 , -0.14814815,  0.51851852],
5         [-0.27777778,  0.11111111,  0.11111111]])
6 >>> np.dot(a, b)
7 array([[ 1.00000000e+00,  5.55111512e-17,  2.22044605e-16],
8         [ 0.00000000e+00,  1.00000000e+00,  5.55111512e-16],
9         [ 1.11022302e-16,  0.00000000e+00,  1.00000000e+00]])
```

2 Библиотека matplotlib в примерах

2.1 Простейший пример

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Создаём массивы
5 t = np.arange(0.0, 2.0, 0.01)
6 s = 1 + np.sin(2*np.pi*t)
7
8 # Рисуем график
9 plt.plot(t, s)
10
```

```
11 # Подписываем оси
12 plt.xlabel('time (s)')
13 plt.ylabel('voltage (mV)')
14
15 # Заголовок рисунка
16 plt.title('About as simple as it gets, folks')
17
18 # Рисовать или нет координатную сетку
19 plt.grid(True)
20
21 # Сохранить в файл (PNG, EPS или PDF)
22 plt.savefig("test.png")
23
24 # Сохраняем в файл с заданным числом пикселей на дюйм
25 plt.savefig("test.png", dpi = 300)
26
27 # Вывести на экран
28 plt.show()
```



Полезные параметры `plt.plot`

- Стиль линии `linestyle` или `ls`

['-' | '--' | '-.' | ':' | 'steps' | ...]

- Толщина линии `linewidth` или `lw`.

- marker

```
['.', ',', 'o', 'v', '^', '<', '>', '1', '2', '3', '4', '8', 's', 'p', '*',  
'h', 'H', '+', 'x', 'D', 'd', '|', '_', 'P', 'X', 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,  
10, 11, ' ', '']
```

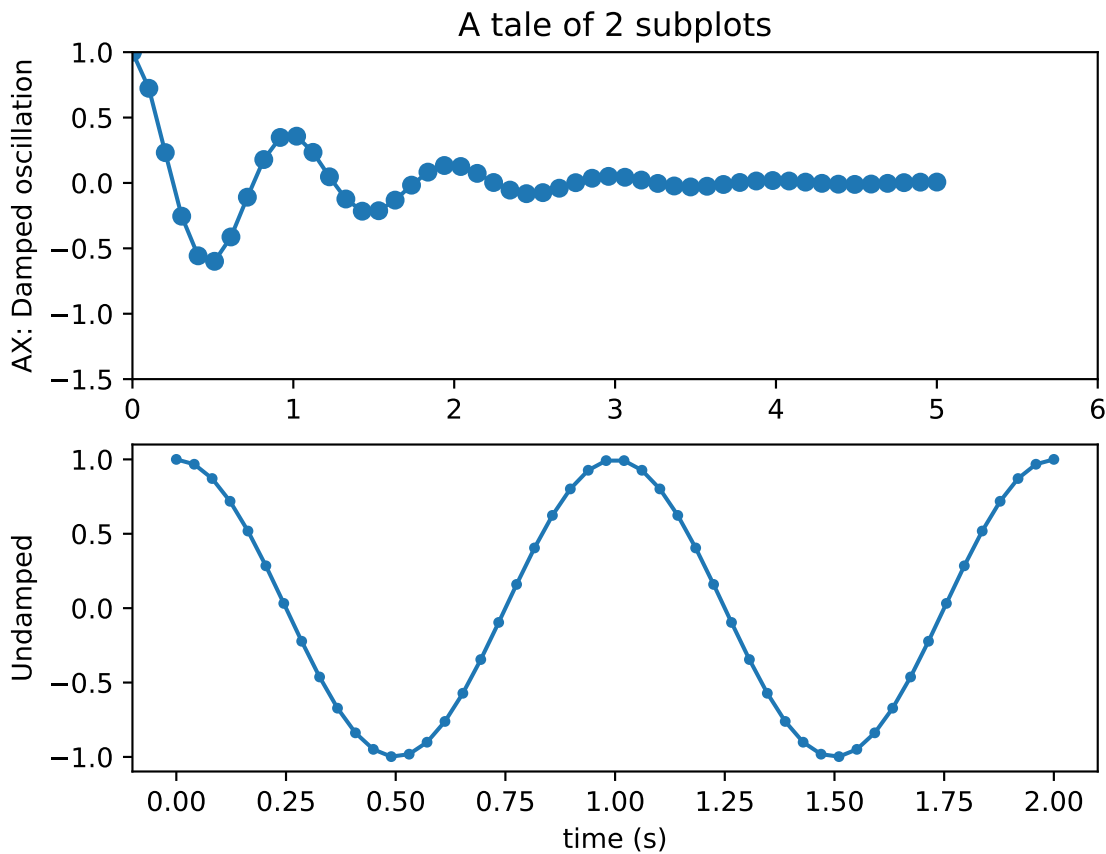
- color — цвет.

Например,

```
1 plt.plot(t, s, lw = 4.5, ls = '-', marker = ':', color='r')  
2 # или color = b,g, и т.д. Или color=(0,1,1) - в формате RGB.
```

2.1.1 Несколько графиков на одном рисунке

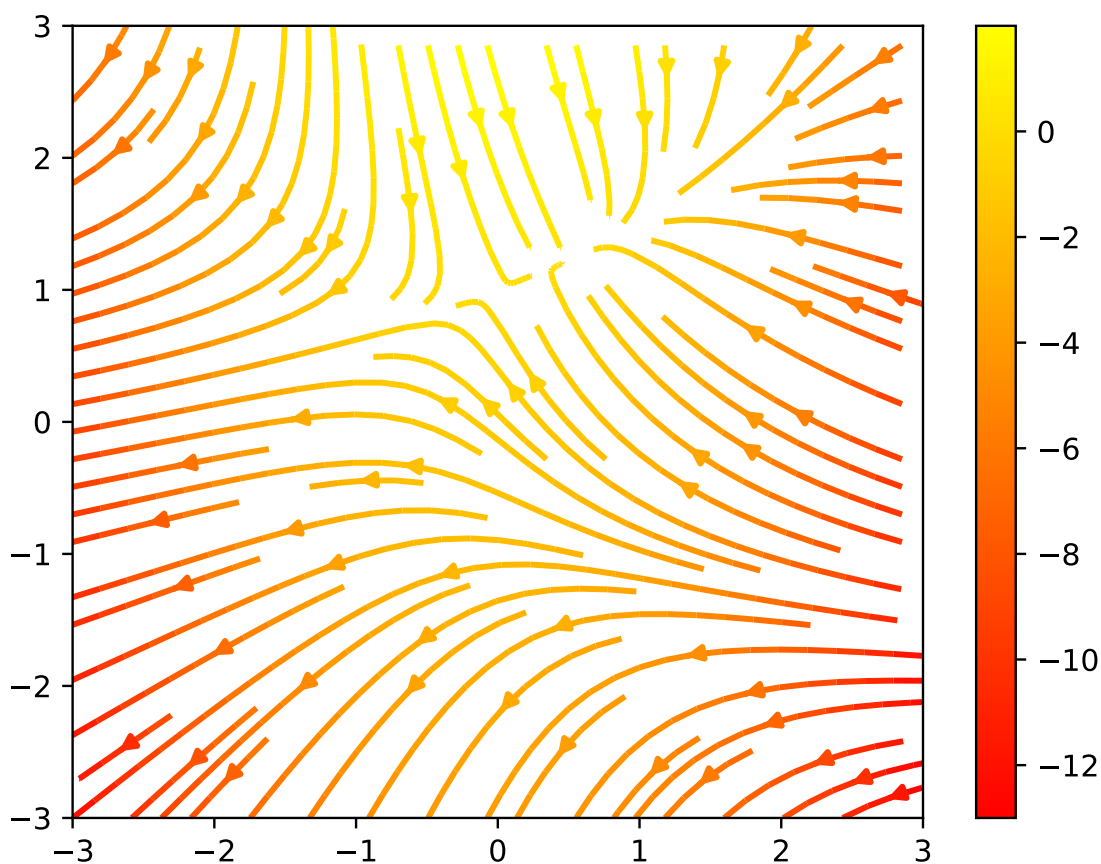
```
1 import numpy as np  
2 import matplotlib.pyplot as plt  
3  
4 # Создаём равномерно распределённые массивы точек  
5 # в заданных промежутках  
6 x1 = np.linspace(0.0, 5.0, num = 50)  
7 x2 = np.linspace(0.0, 2.0)  
8  
9 y1 = np.cos(2 * np.pi * x1) * np.exp(-x1)  
10 y2 = np.cos(2 * np.pi * x2)  
11  
12 # Выбираем первый график  
13 # Первый параметр - общее количество графиков  
14 # Второй и третий параметры отвечают за их  
15 # пространственное расположение друг относительно друга  
16 ax = plt.subplot(2, 1, 1)  
17 plt.plot(x1, y1, 'o-')  
18 plt.title('A tale of 2 subplots')  
19 ax.set_ylabel('AX: Damped oscillation')  
20 ax.set_xlim(0, 6)  
21 ax.set_ylim(-1.5, 1)  
22  
23 plt.subplot(2, 1, 2)  
24 plt.plot(x2, y2, '-.')  
25 plt.xlabel('time (s)')  
26 plt.ylabel('Undamped')  
27  
28 plt.show()
```

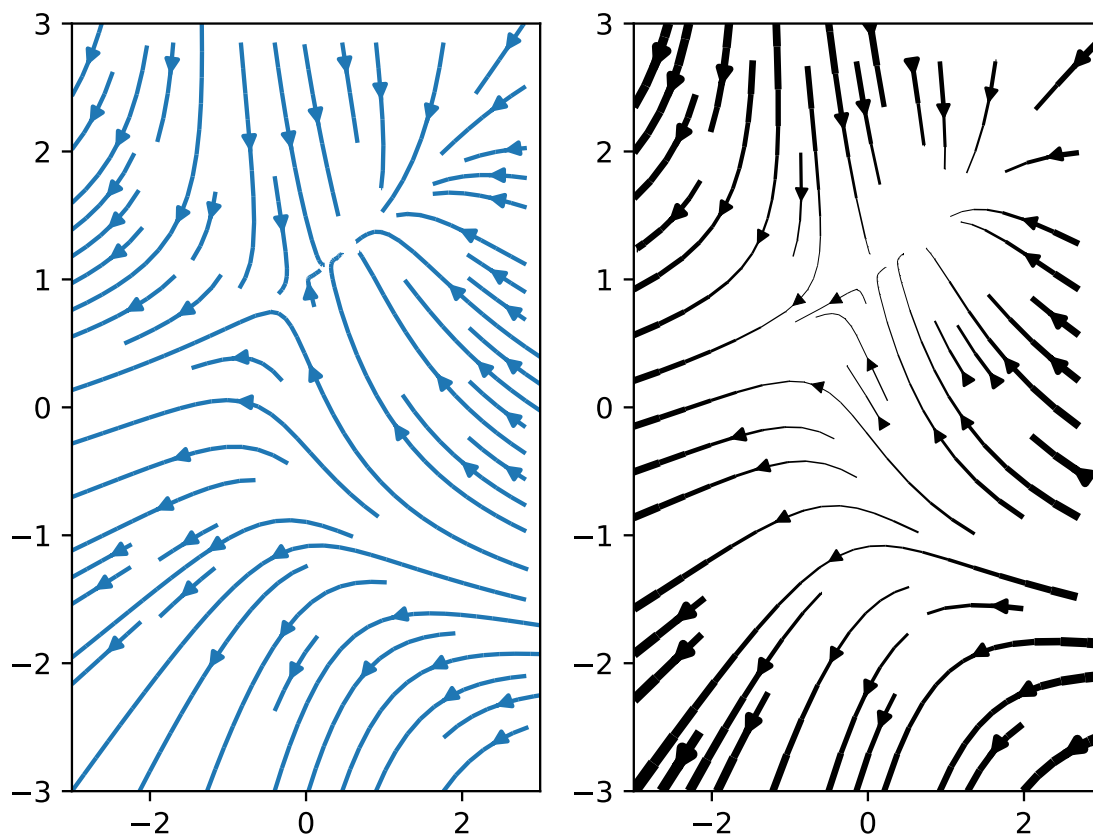


2.2 Streamlines - Трубки тока векторного поля

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Создаём двумерную сетку 100x100 на области [-3, 3] * [-3, 3]
5 Y, X = np.mgrid[-3:3:100j, -3:3:100j]
6
7 # Задаём компоненты функции скорости на сетке
8 U = -1 - X**2 + Y
9 V = 1 + X - Y**2
10
11 # Тогда модуль скорости равен
12 speed = np.sqrt(U*U + V*V)
13
14 # первый рисунок
15 # plt.subplots() возвращает кортеж
16 # классов figure и axis, которые описывают рисунок
17 # и координатные оси соответственно
18 fig0, ax0 = plt.subplots()
19
20 # Первый рисунок
21 strm = ax0.streamplot(X, Y, U, V, color=U, linewidth=2,
22 cmap=plt.cm.autumn)
23
24 # Добавляем цветовой индикатор
```

```
25 fig0.colorbar(strm.lines)
26
27 # Сохраняем первый рисунок
28 fig0.savefig("streamplot.pdf")
29
30
31 # Второй и третий графики в отдельном рисунке
32 # ncols - число столбцов
33 fig1, (ax1, ax2) = plt.subplots(ncols=2)
34 ax1.streamplot(X, Y, U, V, density=[0.5, 1])
35
36 lw = 5*speed / speed.max()
37 ax2.streamplot(X, Y, U, V, density=0.6, color='k',
38 linewidth=lw)
39
40 # Сохраняем второй рисунок
41 fig1.savefig("streamplot1.pdf")
42
43
44 plt.show()
```

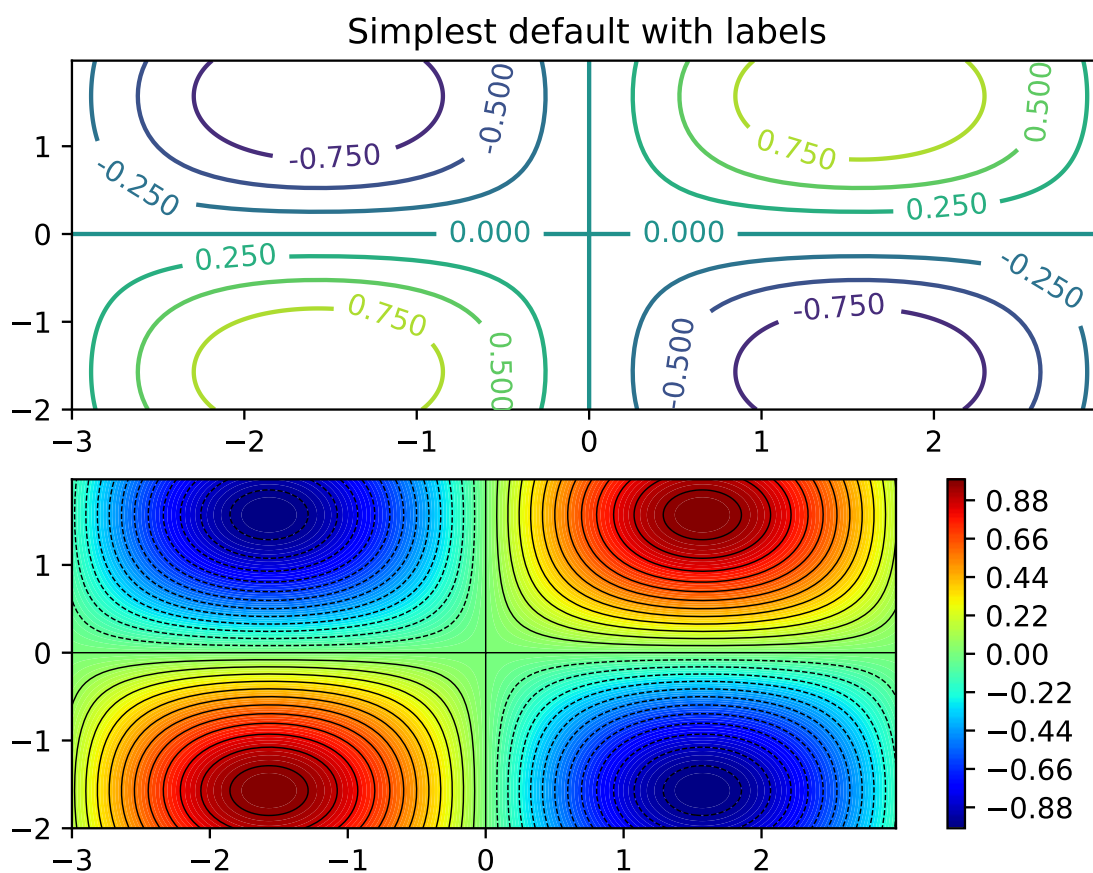




2.3 Контурные

```
1 import matplotlib
2 import numpy as np
3 import matplotlib.cm as cm
4 import matplotlib.pyplot as plt
5
6 delta = 0.025
7
8 # Создаём оси координат
9 x = np.arange(-3.0, 3.0, delta)
10 y = np.arange(-2.0, 2.0, delta)
11
12 # Создаём сетку
13 X, Y = np.meshgrid(x, y)
14
15 # Задаём функцию на сетке
16 Z = np.sin(X)*np.sin(Y)
17
18 # Первый график - рисуем контурные линии
19 plt.subplot(2, 1, 1)
20 CS = plt.contour(X, Y, Z)
21 plt.clabel(CS, inline=1, fontsize=10)
22 plt.title('Simplest default with labels')
23
24 plt.subplot(2, 1, 2)
```

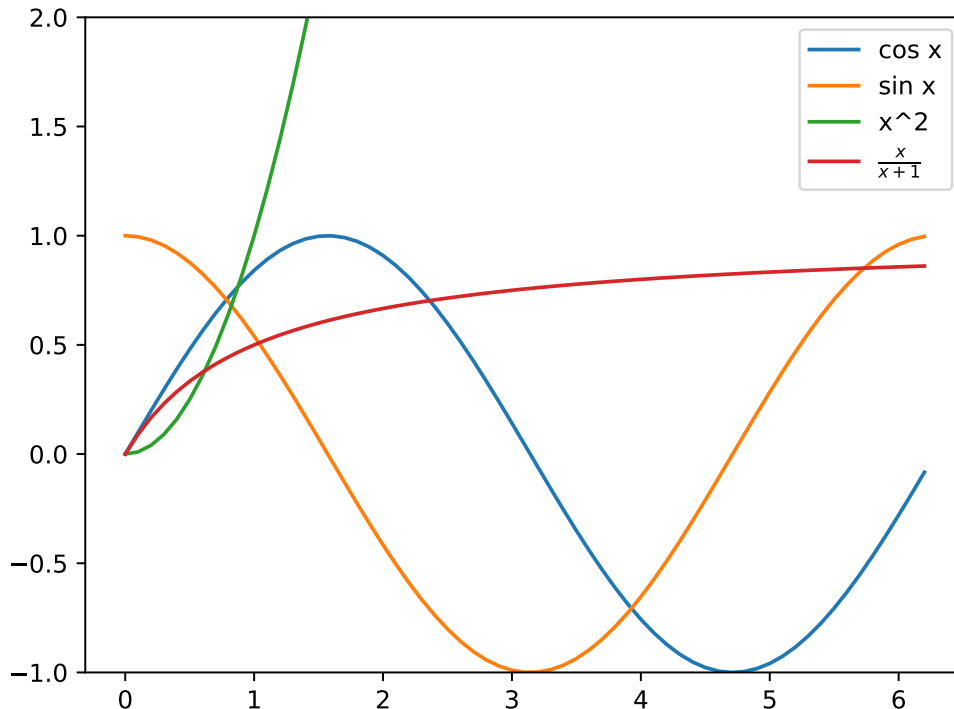
```
25 # Во втором графике рисуем контурные линии - 25 градаций
26 plt.contour(X, Y, Z, 25, linewidths=0.5, colors='k')
27
28 # Рисуем фон - 100 цветовых градаций
29 # cmap=plt.cm.jet - палитра
30 plt.contourf(X, Y, Z, 100, cmap=plt.cm.jet)
31 plt.colorbar()
32
33 plt.savefig("contours.pdf")
34
35 plt.show()
```



2.4 Подписи для функций

```
1 from matplotlib import pyplot as plt
2 import numpy as np
3
4 # Сетка
5 x = np.arange(0, np.pi * 2, 0.1)
6
7 # Четыре функции
8 y = np.sin(x)
9 y2 = np.cos(x)
10 y3 = x * x
11 y4 = x / (x + 1)
```

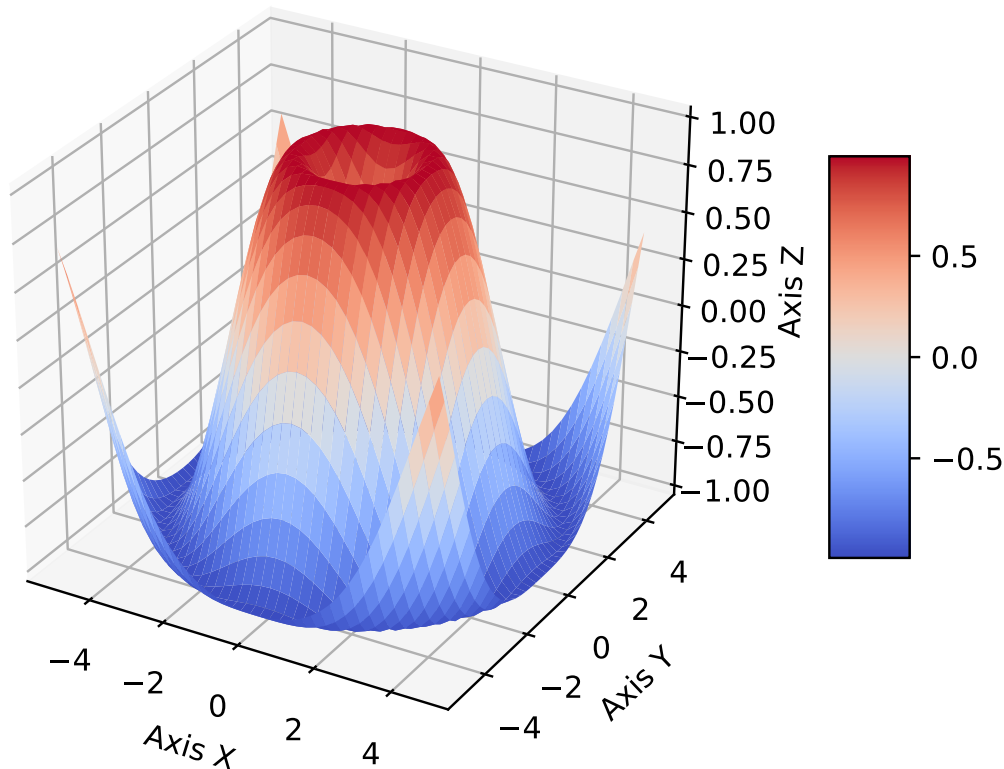
```
12
13 # Получаем класс axis для ограничения
14 # вертикальной оси координат
15 ax = plt.subplot()
16
17 # Устанавливаем пределы для значений по y
18 ax.set_ylim(-1, 2)
19
20 # Рисуем графики
21 plt.plot(x, y, label = 'cos x')
22 plt.plot(x, y2, label = 'sin x')
23 plt.plot(x, y3, label = 'x^2')
24 plt.plot(x, y4, label = '$\\frac{x}{x+1}$')
25 # Символ \ приходится дублировать.
26 # Если хотим писать как в системе LaTeX, то есть без дублирования,
27 # то нужно добавить символ r перед строкой.
28 # Получится label = r'\frac{x}{x+1}'
29
30 # Рисуем подписи
31 plt.legend()
32
33 plt.show()
```



2.5 Рисование поверхностей

```
1 # Axes3D нужен для трёхмерных графиков
2 from mpl_toolkits.mplot3d import Axes3D
```

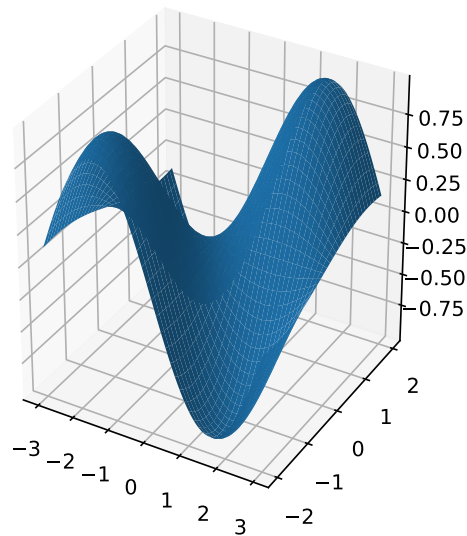
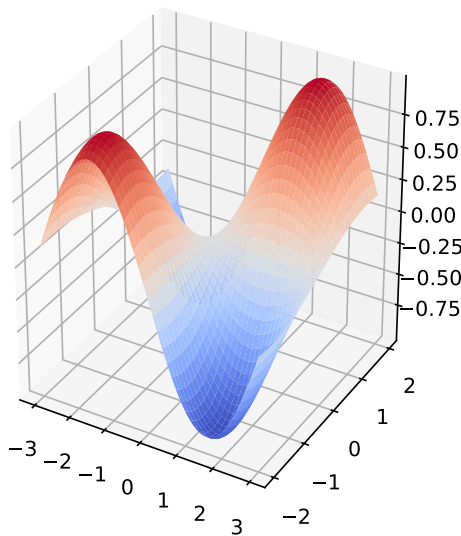
```
3 import matplotlib.pyplot as plt
4 from matplotlib import cm
5 import numpy as np
6
7
8 fig = plt.figure()
9
10 # Для получения класса, описывающего трёхмерные
11 # координатные оси нужно передать параметр
12 # projection='3d' в метод gca() класса Figure
13 ax = fig.gca(projection='3d')
14
15 # Создаём массивы данных.
16 X = np.arange(-5, 5, 0.25)
17 Y = np.arange(-5, 5, 0.25)
18 X, Y = np.meshgrid(X, Y)
19 R = np.sqrt(X**2 + Y**2)
20 Z = np.sin(R)
21
22 # Рисуем поверхность
23 surf = ax.plot_surface(X, Y, Z, cmap=cm.coolwarm)
24
25 # Устанавливаем масштаб по оси z
26 ax.set_zlim(-1.01, 1.01)
27
28 # Ставим подписи к осям
29 ax.set_xlabel('Axis X')
30 ax.set_ylabel('Axis Y')
31 ax.set_zlabel('Axis Z')
32
33 # Рисуем colorbar
34 fig.colorbar(surf, shrink=0.5, aspect=5)
35
36 # Параметр bbox_inches='tight' обрезает лишние поля
37 fig.savefig('surface2.pdf', bbox_inches='tight')
38
39 plt.show()
```



2.6 Рисование нескольких поверхностей на одном рисунке

```
1 import numpy as np
2 from matplotlib import pyplot as plt
3
4 # Нужно для трёхмерных графиков
5 from mpl_toolkits.mplot3d import Axes3D
6 from matplotlib import cm
7
8 delta = 0.025
9
10 # Создаём сетку
11 x = np.arange(-3.0, 3.0, delta)
12 y = np.arange(-2.0, 2.0, delta)
13
14 X, Y = np.meshgrid(x, y)
15
16 # Задаём значения функции на сетке
17 Z = np.sin(X)*np.sin(Y)
18
19 fig = plt.figure(figsize = plt.figaspect(0.5))
20
21 # Добавляем правый график
22 # В отличие от двумерных графиков, которые описываются классом
23 # Axes, трёхмерные описываются классом Axes3D
24 # Для того, чтобы получить его экземпляр, нужно передать
25 # параметр projection='3d' в функцию add_subplot()
```

```
26 ax = fig.add_subplot(1, 2, 1, projection='3d')
27 ax.plot_surface(X, Y, Z, cmap=cm.coolwarm)
28
29 # Добавляем левый график
30 ax = fig.add_subplot(1, 2, 2, projection='3d')
31 ax.plot_surface(X, Y, Z)
32
33 # Параметр bbox_inches='tight' обрезает лишние поля
34 plt.savefig("surface.pdf", bbox_inches='tight')
35
36 plt.show()
```



2.7 Мультики!!!

```
1 import matplotlib as mpl
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import matplotlib.animation as animation
5
6
7 T = 100
8
9 # Выбираем encoder
10 # В данном случае FFMpeg
11 # Для этого он должен быть установлен в системе
12 FFMpegWriter = animation.writers['ffmpeg']
13
14 # fps - число кадров в секунду
15 writer = FFMpegWriter(fps=25)
16
17 # размер шрифта для подписей
18 mpl.rcParams['legend.fontsize'] = 10
19
20 # Нам понадобится класс, описывающий рисунок
```

```
21 fig = plt.figure()
22
23 # Настройки encoder'a
24 # 200 - dpi (число точек на дюйм)
25 writer.setup(fig, "video.avi" , 200)
26
27 # Создаём сетку
28 x = np.arange(0, np.pi * 5, 0.1)
29
30 for t in range(0, T):
31     # Выводим номер кадра на экран для удобства
32     print('Drawing frame %d...' % (t))
33
34     # Заводим функцию на сетке
35     y = np.sin(x - 2 * np.pi * t / T)
36
37     # Рисуем
38     plt.plot(x, y)
39     plt.title("T = %f" % (t))
40
41     # Сохраняем кадр
42     writer.grab_frame()
43
44     # Очищаем рисунок для следующей итерации
45     plt.clf()
```

Список литературы

- [1] Бизли Д. Python. Подробный справочник. СПб: Символ-Плюс. 2010.